

Practical 10

Task 1

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
int main()
{
    struct stat st;
    /* File names */
    char *original = "original.txt";
    char *hardlink = "hardlink.txt";
    char *symlink = "symlink.txt";
    /* Create original file */
    FILE *file = fopen(original, "w");
    if (file == NULL)
    {
        perror("Error creating file");
        return 1;
    }
    fprintf(file, "This is the original file.\n");
    fclose(file);
    printf("Original file created successfully.\n");
    /* Get inode information of original file */
    if (stat(original, &st) == -1)
    {
        perror("stat");
        return 1;
    }
    printf("\n--- Original File ---\n");
    printf("File name      : %s\n", original);
```

```

}
printf("\n--- Original File ---\n");
printf("File name      : %s\n", original);
printf("Inode number   : %lu\n", (unsigned long)st.st_ino);
printf("Link count     : %lu\n", (unsigned long)st.st_nlink);
printf("File size      : %ld bytes\n", (long)st.st_size);
/* Create hard link */
if (link(original, hardlink) == -1)
{
    perror("Error creating hard link");
    return 1;
}
printf("\nHard link created: %s\n", hardlink);
/* Create symbolic link */
if (symlink(original, symlink) == -1)
{
    perror("Error creating symbolic link");
    return 1;
}
printf("Symbolic link created: %s\n", symlink);
/* Information about hard link */
if (stat(hardlink, &st) == -1)
{
    perror("stat hardlink");
    return 1;
}
printf("\n--- Hard Link ---\n");
printf("File name      : %s\n", hardlink);
printf("Inode number   : %lu\n",

```

```

    return 1;
}
printf("\n--- Hard Link ---\n");
printf("File name      : %s\n", hardlink);
printf("Inode number   : %lu\n",
    (unsigned long)st.st_ino);
printf("Link count     : %lu\n",
    (unsigned long)st.st_nlink);
/* Information about symbolic link itself */
if (lstat(symlink, &st) == -1)
{
    perror("lstat symlink");
    return 1;
}
printf("\n--- Symbolic Link ---\n");
printf("File name      : %s\n", symlink);
printf("Inode number   : %lu\n",
    (unsigned long)st.st_ino);
printf("Link count     : %lu\n",
    (unsigned long)st.st_nlink);
printf("\n--- Comparison ---\n");
printf("Original and hard link have the SAME inode.\n");
printf("Symbolic link has a DIFFERENT inode.\n");
return 0;
}

```

Output:

```
root@DESKTOP-VMN932K:~# ./inode
Original File
Inode Number : 65021
Links       : 1

After Creating Links

Original File
Inode Number : 65021
Links       : 2

Hard Link
Inode Number : 65021
Links       : 2

Symbolic Link
Inode Number : 65031

Hard link has the SAME inode as original.
Symbolic link has a DIFFERENT inode.
Symbolic link points to: original.txt
root@DESKTOP-VMN932K:~#
```

Task 2

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
int main()
{
    const char *filename = "sample.txt";
    const char *data = "Hello! This file is accessed using mmap().\n";
    int fd;
    struct stat file_info;
    char *mapped_memory;
    /* -----
       Step 1: Create/Open the file
       ----- */
    fd = open(filename, O_RDWR | O_CREAT, 0644);
    if (fd == -1)
    {
        perror("Error opening file");
        return 1;
    }
    /* -----
       Step 2: Set file size
       ----- */
    if (ftruncate(fd, 1024) == -1)
    {
        perror("Error setting file size");

```

```

    /* -----
       Step 2: Set file size
       ----- */
    if (ftruncate(fd, 1024) == -1)
    {
        perror("Error setting file size");
        close(fd);
        return 1;
    }
    /* Get file information */
    if (fstat(fd, &file_info) == -1)
    {
        perror("fstat");
        close(fd);
        return 1;
    }
    printf("File size: %ld bytes\n", (long)file_info.st_size);
    /* -----
       Step 3: Map file into memory
       ----- */
    mapped_memory = mmap(
        NULL,
        file_info.st_size,
        PROT_READ | PROT_WRITE,
        MAP_SHARED,
        fd,
        0
    );
    if (mapped_memory == MAP_FAILED)

```

```

{
    perror("mmap failed");
    close(fd);
    return 1;
}
printf("\nFile mapped successfully.\n");
printf("Mapped memory address: %p\n",
       (void *)mapped_memory);
/* -----
   Step 4: Write using mmap
   ----- */
strcpy(mapped_memory, data);
printf("\nData written using mmap:\n");
printf("%s", mapped_memory);
/* Make changes visible to the file */
if (msync(mapped_memory, file_info.st_size,
          MS_SYNC) == -1)
{
    perror("msync");
}
/* -----
   Step 5: Read using mmap
   ----- */
printf("\nData read using mmap:\n");
printf("%s", mapped_memory);
/* -----
   Step 6: Unmap memory
   ----- */
if (munmap(mapped_memory, file_info.st_size) == -1)

```

Output:

```

root@Tejasri:~# nano map.c
root@Tejasri:~# gcc map.c -o map
root@Tejasri:~# ./map
File size: 1024 bytes

File mapped successfully.
Mapped memory address: 0x79f5c03ea000

Data written using mmap:
Hello! This file is accessed using mmap().

Data read using mmap:
Hello! This file is accessed using mmap().

File unmapped successfully.
root@Tejasri:~# |

```